

Chương 1. Tổng quan về công nghệ phần mềm

(Nội dung này nằm từ trang 11 đến trang 29 của tài liệu)

1.1. Khái niệm Công nghệ phần mềm (Software Engineering)

1.1.1. Công nghệ phần mềm hướng dự án (Project-based software engineering): Tập trung vào việc phát triển phần mềm tùy chỉnh (custom software) cho một khách hàng cụ thể dựa trên một bản đặc tả yêu cầu và hợp đồng pháp lý.

1.1.2. Công nghệ phần mềm hướng sản phẩm (Product-based software engineering): Tập trung vào việc nhà phát triển tự tìm kiếm cơ hội thị trường và xác định các tính năng để tạo ra phần mềm chung (generic software) cho nhiều khách hàng.

1.2. Phần mềm và lớp phần mềm (Software execution models): : Phần mềm bao gồm các chương trình máy tính và tài liệu liên quan. Các lớp/mô hình thực thi phần mềm bao gồm: chạy độc lập (stand-alone), lai (hybrid) và phần mềm như một dịch vụ (SaaS)

1.2.1. Các mô hình thực thi phần mềm (Software execution models): Dựa trên Hình 1.3 trong tài liệu, bao gồm 3 mô hình chính:

Thực thi độc lập (Stand-alone execution): Chạy hoàn toàn trên máy tính của người dùng

Thực thi lai (Hybrid execution): Một phần chức năng chạy cục bộ, một phần chạy trên máy chủ của nhà cung cấp

Phần mềm dưới dạng dịch vụ (SaaS): Ứng dụng truy cập qua trình duyệt, tính toán chủ yếu diễn ra trên đám mây

1.2.2. Dòng sản phẩm phần mềm và nền tảng (Software product lines and platforms): Dựa trên Bảng 1.1 trong tài liệu:

Dòng sản phẩm phần mềm (Software product line): Các sản phẩm chia sẻ lõi chung nhưng có tùy chỉnh riêng cho khách hàng.

Nền tảng (Platform): Sản phẩm cung cấp các chức năng để xây dựng ứng dụng khác lên trên (ví dụ: Facebook).

1.3. Các công việc trong Công nghệ phần mềm (Software engineering activities): Bao gồm việc hiểu nhu cầu khách hàng, thiết kế kiến trúc, Verifying (xác minh), Testing (kiểm thử) và quản lý mã nguồn.

1.4. Mô hình phát triển phần mềm: Tài liệu nhấn mạnh việc sử dụng các phương pháp **Agile** (linh hoạt) là tiêu chuẩn cho phát triển sản phẩm để đưa sản phẩm ra thị trường nhanh chóng.

1.5. Tầm nhìn sản phẩm (The product vision): Là một tuyên bố ngắn gọn xác định bản chất của sản phẩm, nó khác biệt thế nào với đối thủ và nhắm tới ai.

1.5.1. Các câu hỏi cơ bản (Fundamental questions): Sản phẩm là gì? Đối tượng mục tiêu là ai? Tại sao họ nên mua nó?

1.5.2. Cấu trúc tuyên bố tầm nhìn (Vision template): Sử dụng các từ khóa FOR (CHO), WHO (AI), THAT (MÀ), UNLIKE (KHÁC VỚI) để xác định sự khác biệt.

1.6. Quản lý sản phẩm phần mềm (Software product management): Quản lý lộ trình sản phẩm (roadmap), danh mục công việc (backlog), thiết kế giao diện người dùng và kiểm thử chấp nhận.

1.6.1. Các khía cạnh quan tâm (Technical interactions): Dựa trên Hình 1.5, bao gồm Quản lý lộ trình (Roadmap), Danh mục công việc (Backlog), Thiết kế giao diện (UI) và Kiểm thử chấp nhận.

1.7. Tạo sản phẩm mẫu (Product prototyping): Quá trình phát triển phiên bản sớm của sản phẩm để thử nghiệm ý tưởng, chứng minh tính khả thi và trình diễn cho khách hàng/nhà đầu tư

1.7.1. Trình diễn tính khả thi (Feasibility demonstration): Chứng minh các ý tưởng mới có hoạt động thực tế hay không.

1.7.2. Trình diễn cho khách hàng (Customer demonstration): Sử dụng các kịch bản sử dụng để cho người dùng thấy các tính năng cụ thể sẽ được hiện thực hóa như thế nào.

Chương 2. Công nghệ phần mềm Agile

(Nội dung này nằm từ trang 30 đến trang 59 của tài liệu)

2.1. Phương pháp Agile (Agile methods): tập trung vào việc phân phối chức năng nhanh chóng, phản hồi linh hoạt với các thay đổi và giảm thiểu chi phí quản lý. Nguyên tắc cốt lõi: Con người và tương tác hơn là quy trình và công cụ; phần mềm chạy tốt hơn là tài liệu đầy đủ.

2.1.1. Phát triển gia tăng (Incremental development): Dựa trên việc chia phần mềm thành các tính năng (features). Mỗi bước tăng trưởng sẽ thực hiện, kiểm thử và bàn giao một số ít tính năng cho người dùng để nhận phản hồi sớm.

2.1.2. Tuyên ngôn và Nguyên tắc Agile (The Agile Manifesto and Principles): Tập trung vào cá nhân và tương tác hơn là quy trình; phần mềm chạy tốt hơn là tài liệu đầy đủ.

2.2. Lập trình Extreme (Extreme Programming - XP): Một phương pháp Agile đẩy các quy trình tốt (như phát triển lặp) lên mức "cực đoan". Các thực hành phổ biến: Lập kế

hoạch gia tăng, Phát triển hướng kiểm thử (TDD), Tích hợp liên tục (CI) và Tái cấu trúc mã nguồn (Refactoring).

2.2.1. Các thực hành của XP (XP Practices): Tài liệu liệt kê 10 thực hành cốt lõi (Hình 2.2), trong đó các thực hành phổ biến nhất gồm: Lập kế hoạch gia tăng, Phát hành nhỏ (Small releases), Phát triển hướng kiểm thử (TDD), Tích hợp liên tục (CI) và Tái cấu trúc (Refactoring).

2.2.2. Các thực hành ít phổ biến hơn: Gồm lập trình cặp (Pair programming), quyền sở hữu mã nguồn chung (Collective ownership) và làm việc ở tốc độ bền vững (Sustainable pace).

2.3. Scrum: Một khung quản lý Agile dựa trên các chu kỳ phát triển cố định gọi là Sprint (2-4 tuần). Các vai trò chính gồm Chủ sản phẩm (Product Owner), ScrumMaster và Đội phát triển

2.3.1. Thuật ngữ và Vai trò trong Scrum (Scrum terminology): Định nghĩa các khái niệm then chốt như Chủ sản phẩm (Product Owner), ScrumMaster, Đội phát triển, Sprint và Tăng trưởng sản phẩm có khả năng bàn giao (Potentially shippable product increment).

2.3.2. Chu kỳ Sprint (The Sprint cycle): Quy trình lặp đi lặp lại kéo dài từ 2-4 tuần, bao gồm Lập kế hoạch Sprint (Sprint planning), Họp Scrum hàng ngày (Daily Scrum), Thực thi Sprint và Đánh giá Sprint (Sprint review).

2.3.3. Quản lý Danh mục công việc (Product Backlog management): Quá trình "chải chuốt" (grooming) danh mục bao gồm các hoạt động: Làm mịn (Refinement), Ước lượng (Estimation), Tạo mới (Creation) và Ưu tiên (Prioritization).

2.3.4. Đội ngũ tự tổ chức (Self-organizing teams): Nguyên tắc các nhóm tự đưa ra quyết định về cách phân chia công việc và lịch trình mà không cần quản lý dự án truyền thống chỉ định nhiệm vụ.

Chương 3. Nhu cầu phần mềm: Chức năng, kịch bản và câu chuyện

(Nội dung này nằm từ trang 60 đến trang 91 của tài liệu)

3.1. Người dùng (Personas): Các chân dung nhân vật đại diện cho các nhóm người dùng mục tiêu, giúp đội ngũ phát triển hiểu rõ kỹ năng, trình độ và động lực của họ

3.1.1. Khái niệm Persona: Là những "người dùng tưởng tượng", những chân dung nhân vật đại diện cho các nhóm người dùng mục tiêu mà bạn mong muốn họ sẽ sử dụng sản phẩm.

3.1.2. Lợi ích của Persona: Giúp đội ngũ phát triển hiểu rõ kỹ năng, trình độ và động lực của người dùng, từ đó thiết kế các chức năng và giao diện phù hợp (đóng vai người dùng để đánh giá ý tưởng).

3.1.3. Các khía cạnh mô tả: Bao gồm cá nhân hóa (tên, hình ảnh), công việc (nhiệm vụ liên quan), giáo dục (nền tảng học vấn) và sự liên quan (tại sao họ quan tâm đến sản phẩm).

3.2. Kịch bản (Scenarios): Mô tả mang tính tường thuật về tình huống người dùng sử dụng các tính năng của sản phẩm để đạt được một mục tiêu cụ thể.

3.2.1. Khái niệm Kịch bản: Là một bản tường thuật mô tả tình huống người dùng sử dụng các tính năng của sản phẩm để đạt được một mục tiêu cụ thể.

3.2.2. Các yếu tố quan trọng: Tên kịch bản, các Persona liên quan, mục tiêu tổng thể, những việc cần làm để đạt mục tiêu và cách giải quyết các vấn đề mà hệ thống cũ chưa đáp ứng được.

3.2.3. Viết kịch bản: Nên viết từ góc nhìn của người dùng, sử dụng ngôn ngữ tự nhiên thay vì cấu trúc quá cứng nhắc để kích thích sự sáng tạo trong thiết kế.

3.3. Câu chuyện người dùng (User stories): Câu chuyện người dùng là những bản tường thuật ở mức độ chi tiết (finer-grain) hơn kịch bản, mô tả một điều duy nhất mà người dùng muốn từ hệ thống phần mềm. Định dạng tiêu chuẩn thường là: "Là một <vai trò>, tôi <muốn/cần> <làm một việc gì đó>"

3.3.1. Khái niệm và Cấu trúc: Là các mô tả ngắn gọn, chi tiết hơn kịch bản về một điều duy nhất người dùng muốn. Cấu trúc chuẩn: "Là một <vai trò>, tôi <muốn/cần> <làm gì đó> để <lý do>".

3.3.2. Epic và Story: Phân biệt giữa "Epic" (những câu chuyện lớn cần chia nhỏ) và các story đơn giản có thể thực hiện được trong một phiên phát triển (Sprint).

3.4. Chức năng (Features): Một mẫu chức năng đáp ứng nhu cầu người dùng hoặc hệ thống, thường được xác định từ các kịch bản và câu chuyện người dùng (User stories).

3.4.1. Xác định chức năng (Feature identification): Quá trình trích xuất các mẫu chức năng từ kịch bản và câu chuyện người dùng bằng cách tìm các động từ hành động (như: "sử dụng", "gửi", "cập nhật").

3.4.2. Danh sách chức năng (The feature list): Sản phẩm cuối cùng của quá trình phân tích nhu cầu, bao gồm tên chức năng, mô tả ngắn, các ràng buộc và ghi chú thực hiện.

3.4.3. Tránh "Feature Creep": Cần kiểm soát để số lượng chức năng không tăng quá mức dẫn đến hệ thống phức tạp, gây lỗi và khó sử dụng cho người dùng.

Chương 4. Kiến trúc phần mềm

4.1. Tầm quan trọng của kiến trúc phần mềm (Why is architecture important?): Giải thích cách kiến trúc ảnh hưởng đến các thuộc tính chất lượng như hiệu suất, bảo mật và tính sẵn sàng.

4.2. Thiết kế kiến trúc (Architectural design): Các quyết định đánh đổi (trade-offs) giữa các thuộc tính chất lượng đối nghịch nhau.

4.3. Phân rã hệ thống (System decomposition): Chia nhỏ các thành phần kiến trúc lớn thành các thành phần mịn hơn (mô-đun, đối tượng, dịch vụ) để giảm độ phức tạp.

4.3.1. Các nguyên tắc thiết kế (Design guidelines): Bao gồm Tách biệt các mối quan tâm (Separation of concerns), Giao diện ổn định (Stable interfaces) và Thực hiện một lần (Implement once) để giảm độ phức tạp.

4.3.2. Mô hình kiến trúc phân tầng (Layered architecture): Giới thiệu cấu trúc 5 tầng điển hình cho các ứng dụng web (từ giao diện người dùng đến quản lý giao dịch và cơ sở dữ liệu).

4.3.3. Các mối quan tâm cắt ngang (Cross-cutting concerns): Giải thích về các vấn đề như bảo mật, hiệu suất và độ tin cậy ảnh hưởng đến tất cả các tầng trong hệ thống.

4.4. Kiến trúc phân tán (Distribution architecture): Bao gồm kiến trúc Client-Server, kiến trúc đa tầng (Multi-tier) và kiến trúc hướng dịch vụ (Service-oriented).

4.4.1. Mô hình Client-Server và MVC: Cách tổ chức tương tác giữa máy khách và máy chủ dựa trên mẫu Model-View-Controller.

4.4.2. Kiến trúc đa tầng (Multi-tier architecture): Phân chia cụ thể trách nhiệm giữa máy chủ web, máy chủ ứng dụng và máy chủ cơ sở dữ liệu.

4.4.3. Kiến trúc hướng dịch vụ (Service-oriented architecture): Sử dụng các dịch vụ phi trạng thái (stateless) để tăng khả năng mở rộng và chịu lỗi.

4.5. Các vấn đề kỹ thuật (Technology issues): lựa chọn cơ sở dữ liệu (SQL vs NoSQL), nền tảng phân phối (web vs di động), sử dụng phần mềm nguồn mở và công cụ phát triển.

4.5.1. Lựa chọn cơ sở dữ liệu (Database): So sánh giữa SQL (ACID transactions) và NoSQL (linh hoạt cho dữ liệu phi cấu trúc).

4.5.2. Nền tảng phân phối (Delivery platform): Các cân nhắc khi phát triển cho trình duyệt web so với ứng dụng di động (vấn đề kết nối, năng lượng, giao diện).

4.5.3. Phần mềm nguồn mở và Công cụ phát triển: Lợi ích và rủi ro khi tái sử dụng mã nguồn mở và ảnh hưởng của khung làm việc (framework) đến kiến trúc.

Chương 5. Phần mềm dựa trên đám mây

(Nội dung này nằm từ trang 126 đến trang 159 của tài liệu)

5.1. Ảo hóa và container (Virtualization and containers): Máy ảo (VM) giả lập phần cứng để chạy các hệ điều hành khác nhau; Container là công nghệ ảo hóa nhẹ cấp hệ điều hành giúp triển khai dịch vụ nhanh chóng (ví dụ: Docker).

5.1.1. Máy ảo (Virtual Machines - VMs): Sử dụng một phần mềm gọi là hypervisor để giả lập phần cứng, cho phép chạy nhiều hệ điều hành khác nhau trên cùng một máy chủ vật lý. Tuy nhiên, việc khởi động VM chậm (từ 2-5 phút) do phải tải toàn bộ hệ điều hành.

5.1.2. Container: Là công nghệ ảo hóa nhẹ cấp hệ điều hành, cho phép các máy chủ độc lập chia sẻ chung một hệ điều hành duy nhất. Container khởi động nhanh (vài giây) và tốn ít tài nguyên hơn VM.

5.1.3. Hệ thống Docker: Một hệ thống quản lý container phổ biến cho phép định nghĩa phần mềm trong các "Docker images" để triển khai nhanh chóng và đồng nhất trên mọi môi trường.

5.2. Tất cả như một dịch vụ (Everything as a service - XaaS): Bao gồm Hạ tầng (IaaS), Nền tảng (PaaS) và Phần mềm (SaaS)

5.2.1. Hạ tầng như một dịch vụ (IaaS): Cung cấp các dịch vụ cơ bản như tính toán (compute), mạng (network) và lưu trữ (storage) để người dùng tự thiết lập máy chủ ảo.

5.2.2. Nền tảng như một dịch vụ (PaaS): Cung cấp các thư viện và khung làm việc (frameworks) từ nhà cung cấp đám mây để phát triển ứng dụng có khả năng tự động mở rộng (auto-scaling).

5.3. Phần mềm như một dịch vụ (Software as a service - SaaS): Sản phẩm phần mềm chạy trên đám mây và người dùng truy cập qua trình duyệt hoặc ứng dụng di động theo mô hình đăng ký.

Sản phẩm phần mềm chạy trên đám mây, người dùng truy cập qua trình duyệt web hoặc ứng dụng di động thay vì cài đặt trực tiếp.

Lợi ích: Cải thiện dòng tiền (trả phí thuê bao), quản lý cập nhật tập trung cho mọi khách hàng và dễ dàng thu thập dữ liệu sử dụng để cải tiến sản phẩm.

5.4. Các hệ thống dịch vụ nhiều khách hàng (Multi-tenant) và nhiều thể hiện (Multi-instance): Multi-tenant dùng chung một thể hiện phần mềm và cơ sở dữ liệu cho nhiều khách hàng; Multi-instance cung cấp bản sao hệ thống riêng biệt cho mỗi khách hàng.

5.4.1. Hệ thống nhiều khách hàng (Multi-tenant): Tất cả khách hàng dùng chung một thực thể phần mềm và một cơ sở dữ liệu duy nhất. Dữ liệu được phân tách logic bằng mã định danh khách hàng (tenant ID).

5.4.2. Hệ thống nhiều thể hiện (Multi-instance): Mỗi khách hàng có một bản sao hệ thống và cơ sở dữ liệu riêng biệt chạy trên máy ảo (VM) hoặc container riêng. Cách này an toàn hơn và dễ tùy biến cho từng khách hàng nhưng chi phí vận hành cao hơn.

5.5. Kiến trúc phần mềm dựa trên đám mây (Cloud software architecture): Quyết định về tổ chức cơ sở dữ liệu, khả năng mở rộng (scalability), khả năng chịu lỗi (resilience) và cấu trúc dịch vụ.

Khi thiết kế cho đám mây, cần đưa ra các quyết định về:

Tổ chức cơ sở dữ liệu: Chọn mô hình chia sẻ (multi-tenant) hay riêng biệt (multi-instance) tùy vào nhu cầu bảo mật và tính nhất quán dữ liệu (ACID) của khách hàng.

Khả năng mở rộng và chịu lỗi (Scalability and resilience): Sử dụng cơ chế "scaling out" (thêm nhiều máy chủ nhỏ) thay vì "scaling up" và thiết lập các hệ thống dự phòng (standby systems) ở các vị trí địa lý khác nhau để đảm bảo dịch vụ không bị gián đoạn.

Cấu trúc phần mềm: Lựa chọn giữa kiến trúc nguyên khối (monolithic) cho sản phẩm mẫu hoặc kiến trúc hướng dịch vụ (service-oriented/microservices) để tối ưu hóa việc triển khai trong container trên đám mây.

Chương 6. Kiến trúc Microservices

(Nội dung này nằm từ trang 160 đến trang 194 của tài liệu)

6.1. Microservices: Các dịch vụ quy mô nhỏ, phi trạng thái (stateless), có trách nhiệm đơn nhất và có thể triển khai độc lập

6.1.1. Khái niệm và Đặc điểm: Microservices là các dịch vụ quy mô nhỏ, độc lập, không trạng thái (stateless) và có một trách nhiệm duy nhất. Các đặc điểm chính bao gồm: tự đóng gói (self-contained), nhẹ (lightweight), độc lập về thực thi và triển khai, và hướng theo năng lực nghiệp vụ.

6.1.2. Nguyên tắc Trách nhiệm Đơn nhất (Single Responsibility Principle): Mỗi dịch vụ chỉ nên làm một việc duy nhất và làm thật tốt việc đó. Điều này giúp tăng tính gắn kết (cohesion) và giảm tính phụ thuộc (coupling) giữa các thành phần.

6.1.3. Mã hỗ trợ Microservice (Support code): Ngoài chức năng chính, mỗi microservice cần có mã để quản lý tin nhắn, quản lý lỗi, triển khai giao diện người dùng (UI) và quản lý tính nhất quán của dữ liệu.

6.2. Kiến trúc Microservices: Phong cách kiến trúc xây dựng hệ thống từ các microservices giao tiếp với nhau qua các giao thức nhẹ.

6.2.1. Phong cách kiến trúc: Đây là một phong cách xây dựng hệ thống từ các dịch vụ nhỏ giao tiếp với nhau qua các giao thức nhẹ. Nó giải quyết các vấn đề của kiến trúc nguyên khối (monolithic) như khó mở rộng cục bộ và chậm triển khai thay đổi.

6.2.2. API Gateway: Một thành phần quan trọng đóng vai trò là điểm tiếp xúc duy nhất, chuyển dịch các yêu cầu dịch vụ từ ứng dụng người dùng đến các microservices tương ứng, giúp che giấu sự phức tạp của hệ thống nội bộ.

6.2.3. Các quyết định thiết kế kiến trúc: Bao gồm việc phân rã hệ thống dựa trên khả năng nghiệp vụ (business capabilities), quyết định cách phân tán và chia sẻ dữ liệu, và lựa chọn mô hình giao tiếp giữa các dịch vụ.

6.2.4. Phối hợp dịch vụ (Service coordination): Phân biệt giữa Điều phối (Orchestration) - có một bộ điều khiển trung tâm quản lý quy trình, và Chính hợp (Choreography) - các dịch vụ tự phản ứng dựa trên các sự kiện (events) được phát ra.

6.3. RESTful services: Một bộ quy ước giao tiếp dịch vụ dựa trên giao thức HTTP, sử dụng các động từ (GET, POST, PUT, DELETE) để thao tác trên các tài nguyên (thường định dạng JSON).

6.3.1. Nguyên tắc REST: Dựa trên việc chuyển đổi các biểu diễn của tài nguyên kỹ thuật số giữa máy chủ và máy khách. Các nguyên tắc chính bao gồm sử dụng các động từ HTTP, dịch vụ không trạng thái và tài nguyên được định danh bằng URI.

6.3.2. Các thao tác cơ bản: Sử dụng 4 động từ HTTP chuẩn: GET (Đọc), POST (Tạo), PUT (Cập nhật), và DELETE (Xóa).

6.3.3. Định dạng dữ liệu: Thường sử dụng JSON hoặc XML để biểu diễn tài nguyên trong phần thân của yêu cầu hoặc phản hồi HTTP.

6.4. Triển khai các dịch vụ (Service deployment) : Sử dụng đường ống tích hợp và triển khai liên tục (CI/CD), thường đóng gói dịch vụ trong các container.

6.4.1. Triển khai liên tục (Continuous deployment): Là quy trình tự động hóa hoàn toàn việc đưa mã mới vào sản xuất ngay sau khi thay đổi được xác nhận và vượt qua các bài kiểm thử tự động.

6.4.2. Sử dụng Container và Kubernetes: Các microservices thường được đóng gói trong container (như Docker) để đảm bảo tính nhất quán giữa môi trường phát triển và vận hành. Các hệ thống như Kubernetes giúp quản lý và điều phối hàng trăm container này trong đám mây.

6.4.3. Quản lý phiên bản dịch vụ (Versioned services): Duy trì nhiều phiên bản của một dịch vụ để có thể khôi phục (rollback) nhanh chóng nếu phiên bản mới gặp lỗi hoặc để hỗ trợ các khách hàng chưa kịp cập nhật theo API mới.

Chương 7. Đảm bảo chất lượng phần mềm

(Nội dung này nằm từ trang 231 đến trang 304 của tài liệu)

7.1. Đảm bảo chất lượng mã nguồn (Reliable Programming) :

7.1.1. Tránh lỗi (Fault avoidance): Sử dụng phong cách lập trình giúp giảm thiểu sai sót. Các thực hành cốt lõi bao gồm:

Giảm độ phức tạp (Complexity reduction): Hạn chế sự phụ thuộc giữa các thành phần (coupling) và tăng tính gắn kết trong mỗi thành phần (cohesion).

Sử dụng mẫu thiết kế (Design patterns): Áp dụng các giải pháp đã được chứng minh hiệu quả như mẫu Observer, Facade hay Prototype để tránh việc tự tìm tòi giải pháp mới có thể chứa lỗi.

Tái cấu trúc mã nguồn (Refactoring): Cải thiện cấu trúc mã nguồn mà không thay đổi hành vi bên ngoài để giữ cho chương trình dễ hiểu và dễ bảo trì.

7.1.2. Xác thực đầu vào (Input validation): Kiểm tra định dạng và giá trị của mọi dữ liệu đầu vào để ngăn chặn lỗi vô ý hoặc các cuộc tấn công tiêm mã (injection attacks).

Sử dụng Biểu thức chính quy (Regular expressions) để định nghĩa các mẫu kiểm tra dữ liệu chuỗi một cách nhanh chóng và chính xác.

Thực hiện kiểm tra số (Number checking) để đảm bảo giá trị nằm trong phạm vi hợp lý.

7.1.3. Quản lý lỗi (Failure management): Thiết kế phần mềm sao cho khi có lỗi xảy ra, hệ thống sẽ sụp đổ một cách "an toàn" (fail gracefully).

Sử dụng cơ chế xử lý ngoại lệ (Exception handling) để ngăn chương trình bị dừng đột ngột.

Áp dụng tự động lưu (Auto-save) và ghi nhật ký hoạt động (Activity logging) để giúp người dùng phục hồi công việc sau sự cố.

7.2. Kiểm thử phần mềm (Testing)

7.2.1. Kiểm thử chức năng (Functional testing): Thực thi chương trình với dữ liệu giả lập để phát hiện lỗi và chứng minh phần mềm hoạt động đúng yêu cầu.

Kiểm thử đơn vị (Unit testing): Kiểm tra các hàm hoặc phương thức riêng lẻ ngay trong quá trình phát triển.

Kiểm thử tính năng và hệ thống: Kiểm tra sự tương tác giữa các đơn vị mã nguồn để thực hiện một tính năng và đảm bảo toàn bộ hệ thống hoạt động ổn định.

7.2.2. Kiểm thử tự động (Test automation): Sử dụng các khung kiểm thử (như JUnit) để tự động hóa việc thực thi và báo cáo kết quả kiểm thử, giúp tiết kiệm thời gian và cho phép kiểm thử hồi quy (regression testing) liên tục.

7.2.3. Phát triển hướng kiểm thử (Test-driven development - TDD): Viết các bài kiểm thử tự động trước khi viết mã nguồn thực tế, đảm bảo mã nguồn luôn đáp ứng các bài kiểm thử đã đề ra.

7.2.4. Kiểm thử an toàn (Security testing): Tìm kiếm các lỗ hổng hệ thống mà kẻ tấn công có thể khai thác thông qua cách tiếp cận dựa trên rủi ro (risk-based approach).

7.2.5. Đánh giá mã nguồn (Code reviews): Đồng nghiệp kiểm tra chéo mã nguồn để tìm ra những lỗi hoặc thiếu sót mà các bài kiểm thử tự động có thể bỏ sót, đồng thời chia sẻ kiến thức trong đội ngũ.

Chương 8. DevOps và Quản lý mã nguồn

(Nội dung này nằm từ trang 305 đến trang 338 của tài liệu)

8.1. Quản lý mã nguồn (Code management): Sử dụng các hệ thống như Git để quản lý các phiên bản mã, hỗ trợ làm việc nhóm song song thông qua cơ chế nhánh (branching) và hợp nhất (merging).

8.1.1. Khái niệm và tầm quan trọng: Là một tập hợp các thực hành được hỗ trợ bởi phần mềm để quản lý một mã nguồn đang tiến hóa, đảm bảo các thay đổi từ nhiều nhà phát triển không xung đột với nhau.

8.1.2. Các tính năng cơ bản: Bao gồm chuyển giao mã (code transfer), lưu trữ và truy xuất phiên bản (version storage and retrieval), phân nhánh và hợp nhất (branching and merging), và cung cấp thông tin phiên bản.

8.1.3. Hệ thống Git: Git là một hệ thống quản lý phiên bản phân tán (DVCS) phổ biến nhất hiện nay. Thay vì chỉ giữ bản sao các file đang làm việc, Git duy trì một bản sao (clone) của toàn bộ kho lưu trữ (repository) trên máy tính của mỗi nhà phát triển.

8.1.4. Phân nhánh và Hợp nhất (Branching and Merging): Cho phép các nhà phát triển làm việc song song trên các nhánh độc lập mà không can thiệp vào mã nguồn chính (master branch). Khi hoàn thành và kiểm thử xong, các nhánh này sẽ được hợp nhất lại vào nhánh chính.

8.2. Tự động hóa DevOps (DevOps automation): Tự động hóa tích hợp liên tục (CI), chuyển giao liên tục (CD) và hạ tầng dưới dạng mã (IaC)

8.2.1. Nguyên tắc "Tự động hóa mọi thứ": DevOps dựa trên việc tự động hóa các quy trình tích hợp, triển khai và giải phóng phần mềm để giảm thời gian và chi phí, đồng thời tăng độ tin cậy.

8.2.2. Tích hợp liên tục (Continuous Integration - CI): Quy trình xây dựng và kiểm thử phiên bản thực thi của hệ thống ngay khi có thay đổi được đẩy lên kho lưu trữ chung. Điều này giúp phát hiện lỗi sớm và luôn có một hệ thống sẵn sàng hoạt động.

8.2.3. Chuyển giao và Triển khai liên tục (Continuous Delivery and Deployment):

Chuyển giao liên tục (CD): Đảm bảo mã nguồn sau khi thay đổi luôn ở trạng thái sẵn sàng để bàn giao cho khách hàng

Triển khai liên tục: Một bước xa hơn của CD, trong đó mọi thay đổi vượt qua các bài kiểm thử tự động sẽ được triển khai trực tiếp lên môi trường sản xuất (thường áp dụng cho phần mềm dựa trên đám mây).

8.2.4. Hạ tầng dưới dạng mã (Infrastructure as Code - IaC): Sử dụng các mô hình máy có thể đọc được để định nghĩa hạ tầng (mạng, máy chủ, phần mềm cài đặt...) và sử dụng các công cụ cấu hình như Puppet, Chef để tự động thiết lập môi trường thực thi.

8.3. Đánh giá DevOps (DevOps measurement): Sử dụng các phép đo như tần suất triển khai, tỷ lệ triển khai thất bại và thời gian trung bình để khôi phục (MTTR) để cải thiện quy trình.

8.3.1. Tầm quan trọng của phép đo: Để cải tiến quy trình DevOps, cần thu thập và phân tích dữ liệu về sản phẩm và quy trình theo thời gian.

8.3.2. Các loại phép đo chính:

Số liệu quy trình (Process metrics): Bao gồm tần suất triển khai (deployment frequency), tỷ lệ triển khai thất bại, thời gian trung bình để khôi phục (MTTR) và thời gian chờ từ khi phát triển đến khi triển khai (lead time).

Số liệu dịch vụ (Service metrics): Thu thập dữ liệu về hiệu suất, độ tin cậy của phần mềm và sự chấp nhận của khách hàng (ví dụ: số lượng khiếu nại, số lượng khách hàng mới).

8.3.3. Sử dụng bảng điểm (Scorecard) và Bảng điều khiển (Dashboard): Dữ liệu thu thập nên được trình bày trực quan thông qua các bảng điều khiển để giúp đội ngũ theo dõi xu hướng dài hạn và đưa ra quyết định cải tiến quy trình.

Chương 9. An toàn và Tính riêng tư

(Nội dung này nằm từ trang 195 đến trang 230 của tài liệu)

9.1. Tấn công và phòng thủ (Attacks and defenses): Các loại tấn công phổ biến như tiêm mã SQL, cross-site scripting (XSS), chiếm đoạt phiên (session hijacking) và từ chối dịch vụ (DoS)

9.1.1. Các loại tấn công phổ biến:

Tấn công tiêm mã (Injection attacks): Bao gồm tấn công tràn bộ đệm (C/C++) và tiêm mã SQL (SQL poisoning), nơi kẻ tấn công chèn lệnh độc hại vào các trường nhập liệu.

Tấn công kịch bản chéo trang (Cross-site scripting - XSS): Kẻ tấn công thêm mã JavaScript độc hại vào trang web để đánh cắp thông tin hoặc cookie phiên của người dùng.

Chiếm đoạt phiên (Session hijacking): Kẻ tấn công lấy được cookie phiên hợp lệ để mạo danh người dùng.

Tấn công từ chối dịch vụ (DoS/DDoS): Làm hệ thống quá tải bằng hàng loạt yêu cầu giả mạo khiến người dùng thực không thể truy cập.

Tấn công vét cạn (Brute force): Thử mọi kết hợp mật khẩu có thể cho đến khi tìm ra mật khẩu đúng.

9.1.2. Chiến lược phòng thủ: Sử dụng xác thực đầu vào (input validation) để chặn mã độc, mã hóa lưu lượng truy cập (HTTPS), và thiết lập thời gian chờ (timeout) ngắn cho các phiên làm việc.

9.2. Xác thực (Authentication): Xác minh danh tính người dùng dựa trên mật khẩu, thiết bị sở hữu hoặc sinh trắc học

9.2.1. Các yếu tố xác thực: Dựa trên những gì người dùng biết (mật khẩu), những gì họ có (thiết bị di động, mã gửi qua SMS) hoặc đặc điểm sinh trắc học (vân tay, khuôn mặt).

9.2.2. Danh tính liên hợp (Federated identity): Cho phép người dùng đăng nhập bằng tài khoản của các dịch vụ tin tưởng bên thứ ba như Google hoặc Facebook (sử dụng giao thức OAuth).

9.2.3. Xác thực đa yếu tố (Multi-factor authentication): Kết hợp nhiều phương thức để tăng cường bảo mật.

9.3. Phân quyền (Authorization): Kiểm soát quyền truy cập vào tài nguyên hệ thống dựa trên danh tính đã xác thực, thường dùng danh sách kiểm soát truy cập (ACL).

9.3.1. Chính sách kiểm soát truy cập: Định nghĩa các quy tắc về việc ai có quyền truy cập vào tài nguyên nào và các thao tác được phép thực hiện (đọc, sửa, xóa).

9.3.2. Danh sách kiểm soát truy cập (ACLs): Các bảng liên kết người dùng hoặc nhóm người dùng với các quyền hạn cụ thể trên tài nguyên hệ thống.

9.4. Mã hóa (Encryption): Chuyển đổi tài liệu thành dạng không thể đọc được nếu không có khóa bí mật (mã hóa đối xứng và bất đối xứng).

9.4.1. Mã hóa đối xứng và bất đối xứng:

Đối xứng (Symmetric): Dùng chung một khóa cho cả mã hóa và giải mã (như AES).

Bất đối xứng (Asymmetric): Sử dụng cặp khóa công khai (public) và riêng tư (private) (như TLS/SSL).

9.4.2. Trạng thái dữ liệu cần mã hóa: Dữ liệu đang truyền đi (in transit), dữ liệu đang lưu trữ (at rest), và dữ liệu đang được sử dụng (in use).

9.4.3. Quản lý khóa (Key management): Quy trình đảm bảo các khóa mã hóa được tạo, lưu trữ và thu hồi một cách an toàn thông qua các hệ thống KMS.

9.5. Tính riêng tư (Privacy): Liên quan đến việc thu thập và sử dụng thông tin cá nhân hợp lý, tuân thủ các đạo luật bảo vệ dữ liệu như GDPR.

9.5.1. Khái niệm và đạo luật: Quyền của cá nhân đối với thông tin cá nhân của họ, được bảo vệ bởi các đạo luật như Quy định bảo vệ dữ liệu chung (GDPR) của EU.

9.5.2. Các nguyên tắc bảo vệ dữ liệu: Bao gồm sự nhận thức và kiểm soát của người dùng, mục đích thu thập rõ ràng, sự đồng ý (consent), và thời hạn lưu trữ dữ liệu.

9.5.3. Trách nhiệm của nhà phát triển: Thiết lập chính sách riêng tư minh bạch, cung cấp bảng điều khiển quyền riêng tư (privacy dashboard) để người dùng tự quản lý thông tin chia sẻ.